

Phase 5
Project Development Phase

Phase 5: Project Development Phase

Covers initializing the Node.js environment, organizing the MVC folder structure in VS Code, and building the Express.js server. It details configuring the MongoDB connection using Mongoose (db.js), defining data schemas (User.js and Blog.js), setting up authentication and error middleware, and integrating Google Gemini AI services.

Creating project folder

1. Create a new folder named **AI StudyBuddy** .
2. Open the project folder in **Visual Studio Code**.
3. Initialize the project using Node.js and install the required dependencies.

Project Initialization:

Open the project folder in the VS Code terminal and execute:

```
npm init -y
```

Install the required dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

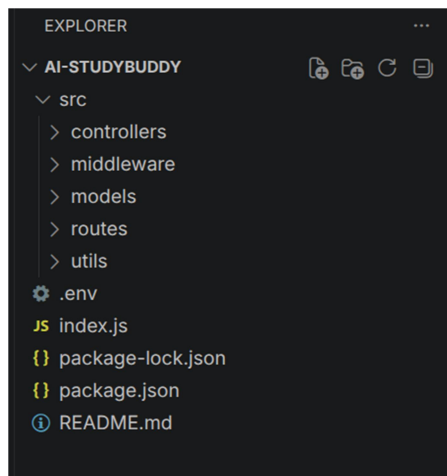
Install Nodemon for development:

```
npm install --save-dev nodemon
```

Create the Following Files

- index.js
- .env
- package.json
- .gitignore

Create the Following Folder Structure



Environment Configuration

Create a **.env** file in the project root and configure the required environment variables.

PORT=5000

MONGO_URI=your_mongodb_connection_string

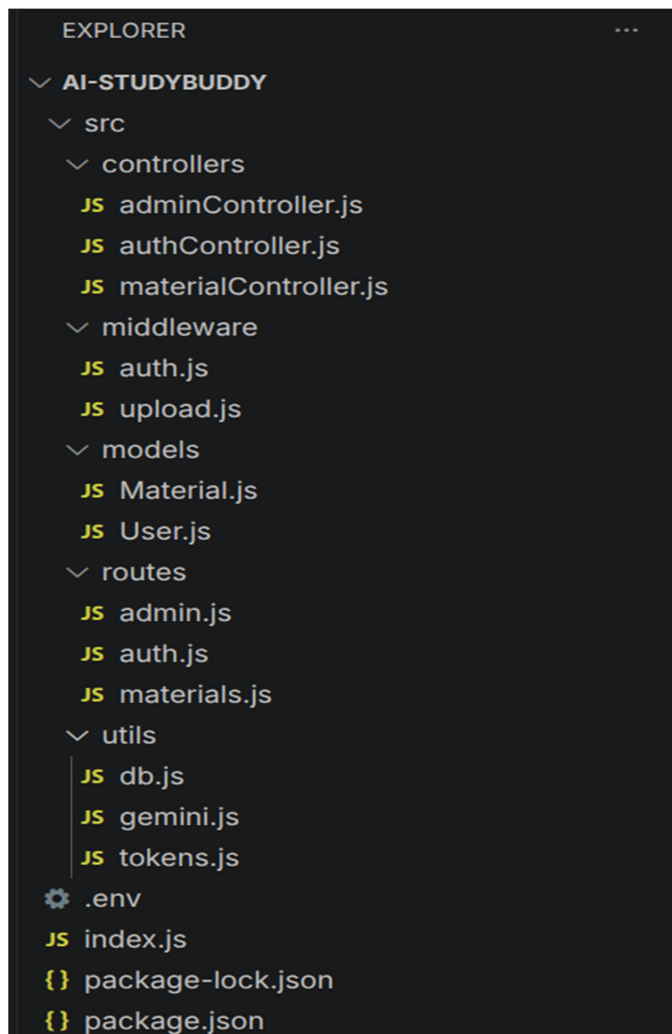
JWT_SECRET=your_jwt_secret

GEMINI_API_KEY=your_gemini_api_key

Backend Development

The AI Studybuddy Backend Is Developed Using Node.Js, Express.Js, And Mongodb, Following A Modular Architecture. The Application Separates Routing, Controllers, Middleware, Models, And Utility Modules To Improve Code Maintainability And Scalability. The Backend Exposes Secure REST Apis For Authentication, Study Material Management, AI-Powered Content Generation, And Administrative Operations.

Backend Structure



Core System Script

Implementations

1. Application Entry Point

(index.js)

The index.js file serves as the main entry point of the application. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful database connection.

```
JS index.js > ...
1  require("dotenv").config();
2  require("express-async-errors");
3
4  const express = require("express");
5  const cors = require("cors");
6  const connectDB = require("../src/Utils/db");
7
8  const app = express();
9
10 // Middleware
11 app.use(express.json());
12 app.use(cors({ origin: process.env.CLIENT_URL || "http://localhost:3000" }));
13
14 // Routes
15 app.use("/api/auth", require("../src/routes/auth"));
16 app.use("/api/materials", require("../src/routes/materials"));
17 app.use("/api/admin", require("../src/routes/admin"));
18
19 // Health check
20 app.get("/", (req, res) => res.json({ message: "AI StudyBuddy API is running" }));
21
22 // Global error handler
23 app.use((err, req, res, next) => {
24   console.error(err.message);
25   const status = err.status || 500;
26   res.status(status).json({ message: err.message || "Something went wrong" });
27 });
28
29 const PORT = process.env.PORT || 5000;
30
31 connectDB().then(() => {
32   app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
33 });
34
```

2. Database Connection

(src/utils/db.js)

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

```
src > utils > JS db.js > ...
1  const mongoose = require("mongoose");
2  const dotenv = require("dotenv");
3
4  dotenv.config();
5
6  const connectDB = async () => {
7    const conn = await mongoose.connect(process.env.MONGO_URI);
8    console.log(`MongoDB connected: ${conn.connection.host}`);
9  };
10
11 module.exports = connectDB;
12
```

4. Authentication

Middleware

(src/middleware/auth.js)

The authentication middleware verifies JWT access tokens for protected routes. It authenticates users before granting access to secured APIs and includes role-based authorization to restrict administrative operations.

```

src > middleware > JS auth.js > ...
1  const jwt = require("jsonwebtoken");
2
3  const protect = (req, res, next) => {
4      const authHeader = req.headers.authorization;
5      if (!authHeader || !authHeader.startsWith("Bearer ")) {
6          return res.status(401).json({ message: "Not authenticated" });
7      }
8
9      const token = authHeader.split(" ")[1];
10
11     try {
12         const decoded = jwt.verify(token, process.env.JWT_ACCESS_SECRET);
13         req.user = decoded;
14         next();
15     } catch {
16         return res.status(401).json({ message: "Invalid or expired token" });
17     }
18 };
19
20 const adminOnly = (req, res, next) => {
21     if (req.user.role !== "admin") {
22         return res.status(403).json({ message: "Admins only" });
23     }
24     next();
25 };
26
27 module.exports = { protect, adminOnly };
28

```

AI Integration (src/utls/gemini.js)

This utility integrates the backend with the Google Gemini 2.5 Flash API. It sends prompts to the AI model and returns generated responses for summaries, flashcards, quizzes, and study plans.

```

src > utls > JS gemini.js > ...
1  const { GoogleGenerativeAI } = require("@google/generative-ai");
2
3  const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
4  const model = genAI.getGenerativeModel({ model: "gemini-2.5-flash" });
5
6  const askGemini = async (prompt) => {
7      const result = await model.generateContent(prompt);
8      return result.response.text();
9  };
10
11 module.exports = { askGemini };
12

```

File Upload Middleware (src/middleware/upload.js)

The upload middleware handles study material uploads and validates incoming files before they are processed by the backend.

```

src > middleware > JS upload.js > ...
1  const multer = require("multer");
2  const path = require("path");
3
4  const storage = multer.diskStorage({
5    destination: (req, file, cb) => cb(null, "uploads/"),
6    filename: (req, file, cb) => {
7      const unique = Date.now() + "-" + Math.round(Math.random() * 1e9);
8      cb(null, unique + path.extname(file.originalname));
9    },
10 });
11
12 const fileFilter = (req, file, cb) => {
13   const allowed = [".txt", ".md", ".pdf"];
14   const ext = path.extname(file.originalname).toLowerCase();
15   if (allowed.includes(ext)) {
16     cb(null, true);
17   } else {
18     cb(new Error("Only .txt, .md, and .pdf files are allowed"));
19   }
20 };
21
22 const upload = multer({ storage, fileFilter, limits: { fileSize: 5 * 1024 * 1024 } });
23
24 module.exports = upload;

```

Epic 4: Database Configuration

The AI StudyBuddy backend uses MongoDB as its primary NoSQL database and Mongoose ODM for data modeling and schema validation. MongoDB stores user information, uploaded study materials, and AI-generated learning resources in a structured manner. Mongoose simplifies database operations by providing schemas, validation rules, and efficient query handling.

Configure MongoDB

Install Mongoose using the following command:

```
npm install mongoose
```

Create Database Connection

Before performing any database operations, establish a connection between the backend and MongoDB using Mongoose. The connection module is initialized during server startup to ensure reliable communication with the database.

Mongoose Schema Definitions

1. User Schema (models/[User.js](#))

The User schema stores authentication and profile information for registered users. It includes fields such as name, email, password, and role. Passwords are securely encrypted before being stored in the database.

```
src > models > JS User.js > ...
1  const mongoose = require("mongoose");
2  const bcrypt = require("bcryptjs");
3
4  const userSchema = new mongoose.Schema(
5    {
6      name: { type: String, required: true, trim: true },
7      email: { type: String, required: true, unique: true, lowercase: true },
8      password: { type: String, required: true },
9      role: { type: String, enum: ["student", "admin"], default: "student" },
10    },
11    { timestamps: true }
12  );
13
14  // Hash password before saving
15  userSchema.pre("save", async function (next) {
16    if (!this.isModified("password")) return next();
17    this.password = await bcrypt.hash(this.password, 10);
18    next();
19  });
20
21  userSchema.methods.matchPassword = function (password) {
22    return bcrypt.compare(password, this.password);
23  };
24
25  module.exports = mongoose.model("User", userSchema);
26
```

2. Study Material Schema (models/Material.js)

The Study Material schema stores the learning content uploaded by users. This content acts as the input for AI-powered features such as summaries, quizzes, flashcards, and study plans.

```

src > models > JS Material.js > ...
1  const mongoose = require("mongoose");
2
3  const materialSchema = new mongoose.Schema(
4    {
5      user: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },
6      title: { type: String, required: true },
7      content: { type: String, required: true }, // raw text content
8      filename: { type: String }, // original uploaded file name
9      summary: { type: String },
10     flashcards: [
11       {
12         question: String,
13         answer: String,
14       },
15     ],
16     quiz: [
17       {
18         question: String,
19         options: [String],
20         answer: String,
21       },
22     ],
23     studyPlan: { type: String },
24   },
25   { timestamps: true }
26 );
27
28 module.exports = mongoose.model("Material", materialSchema);
29

```

Database Collections

The AI StudyBuddy database consists of the following collections:

Users – Stores user account and authentication details.

Study Materials – Stores uploaded learning materials.

API Route Interface Endpoints

The backend exposes RESTful APIs to interact with the database and AI services.

1. Authentication APIs

Handles user registration, login, JWT authentication, and protected route access.

2. Study Material APIs

Allows users to upload, retrieve, update, and manage study materials.

3. AI Feature APIs Provides endpoints for:

AI Summary Generation

AI Flashcard Generation

AI Quiz Generation

AI Study Plan Generation

4. Administrative APIs

Provides administrator-only endpoints for managing users and monitoring system activities.

Epic 5: Project Execution

The AI StudyBuddy backend can be executed locally using Node.js. Before running the application, ensure that MongoDB is running and the required environment variables are configured correctly. The backend exposes REST APIs that can be tested using Postman or Thunder Client.

Running the Backend Server

Install all required dependencies:

```
npm install
```

Start the development server:

```
npm start
```

After successful execution, the server connects to MongoDB and starts listening for incoming API requests.

GitHub: [Code Link](#)